

CodeLens — Executable Checkpoint Roadmap

Context

CodeLens has a **complete strategic planning layer** (six founding docs: CORE, STRATEGY, FOUNDATION, EXPERIENCE, ARCHITECTURE, ROADMAP) and a **frozen graph schema** (``schema.py``). What it does *not* have is the executable layer beneath ``ROADMAP.md``: the granular, verifiable "do this → prove it → then this" sequence that turns empty scaffolding into a shipped, validated product and a startup with traction.

The code today is **bare scaffolding**: ``main.py`` is a FastAPI health check; ``parser/``, ``graph/``, ``ai/``, ``risk/``, ``api/``, ``chat/`` are empty ``__init__.py`` files. There is no parser, graph store, pipeline, query engine, or frontend. Two drifts already contradict the team's own ARCHITECTURE rulings and must be corrected in Checkpoint 0:

- ``backend/app/core/config.py`` hardcodes Postgres + Redis + Neo4j from day one (ARCHITECTURE §"Two corrections" says **SQLite + NetworkX now**, the others *when load justifies each*).
- ``backend/requirements.txt`` pulls in Celery, ChromaDB, Kubernetes, LangGraph (the "five databases for zero users" trap STRATEGY §7 explicitly warns against) — while **``networkx``**, the MVP traversal engine, is missing entirely.

The intended outcome: a single authoritative ``PLANNING/CHECKPOINTS.md`` that a solo founder can execute top-to-bottom, where every checkpoint carries a concrete deliverable, a hard acceptance gate (the "honest-theater" discipline applied to progress itself), and its strategic tie-back. It threads the business track (smoke test, ICP interviews, kill/continue gate, pricing, launch) through the engineering spine at the exact moments STRATEGY §7 prescribes — execution-first, startup woven in.

This plan file describes the **structure and content** of that document so approval is meaningful. Executing the plan = writing ``PLANNING/CHECKPOINTS.md``. It does **not** modify any code — Checkpoint 0's fixes are *documented as the first checkpoint*, to be executed only in a follow-on session.

Deliverable

File: ``PLANNING/CHECKPOINTS.md`` — new, slots directly under ``ROADMAP.md`` in the existing doc stack.

- **Document conventions**** (stated once at the top of the doc):
- Every checkpoint = ****Deliverable**** (what concretely exists when done) + ****Gate**** (the observable test that proves it — "done" means the gate passes, nothing softer) + ****Why**** (the strategic tie to a founding doc).
 - Checkpoints are numbered ``CP-N.M``; business checkpoints are ``B-N``. Bold ****MILESTONE GATES**** mark the ARCHITECTURE M1–M5 acceptance moments and the STRATEGY kill/continue gate.
 - A "critical path vs. parallel" note so the founder knows what blocks what.
 - Rough week ranges as **guidance**, explicitly labeled non-binding.

Checkpoint structure to be written

Stage 0 — Foundation realignment *(Week 0)*

- ****CP-0.1 Realign scaffolding to SQLite-first.**** Rewrite ``config.py`` to a single ``SQLITE_PATH`` + optional keys, delete the Neo4j/Postgres/Redis hardcoding. Trim ``requirements.txt`` to the MVP set (tree-sitter[-python/-javascript], GitPython, radon, fastapi, pydantic, anthropic/openai, ****add `networkx` + `sqlite-vss`/`faiss-cpu`****; drop celery/chromadb/kubernetes/langgraph until Stage 6+). One source of truth for the schema (``backend/app/graph/schema.py``; ``PLANNING/schema.py`` becomes a doc copy or is removed). Add ruff/mypy/pytest config + a minimal CI file.
 - ***Gate:*** ``pytest`` green, ``uvicorn app.main:app`` boots, ``/health`` responds, ruff clean, ``import networkx`` works.
- ****CP-0.2 Golden-fixture harness.**** ``fixtures/`` of tiny hand-verified repos with expected node/edge counts (ARCHITECTURE §2).
 - ***Gate:*** harness loads a fixture and asserts its expected graph; runs in CI.
- ****B-0 Smoke test (parallel).**** Landing page + waitlist, problem posts in 5 ICP communities (STRATEGY §7 Phase 0).
 - ***Gate:*** resonance signal measured before heavy build (comments/waitlist signups).

Stage 1 — The core asset · M1, no AI *(Weeks 1–3)*

- ****CP-1.1 Ingestion**** → ``RepoSnapshot`` (shallow clone, size-cap, timeout, language census).
 - ***Gate:*** clones 3 real repos incl. CodeLens itself; census correct.
- ****CP-1.2 Python parser → Layer A facts**** (CONTAINS/IMPORTS/CALLS/INHERITS, complexity via radon, entrypoint detection).
 - ***Gate:*** golden fixtures pass; all emitted nodes/edges schema-valid.
- ****CP-1.3 Call resolution + confidence flags**** (``resolved`/`heuristic`/`dynamic_unknown``) — the named-hard problem.
 - ***Gate:*** hand-verify 20 CALLS edges on a real repo → ≥90% of ``resolved`` correct.
- ****CP-1.4 GraphStore interface + SQLite persistence + NetworkX load + checkpointed pipeline**** (``cloned`→`parsed`→`graph_built``, idempotent, hash-keyed).

- **Gate:* build → persist → reload → traverse CodeLens's own graph; unchanged re-run skips work.
- ****CP-1.5 Git-lite temporal pass**** (churn, author_count, last_modified via one ``git log --numstat``).
 - **Gate:* churn spot-checks match ``git log``.
- ****CP-1.6 Crude visual dump**** (GraphViz/JSON) to eyeball correctness.
 - *****MILESTONE GATE M1:***** runs on 3 real repos, golden tests pass, 20-edge spot check ≥90%.
- ****B-1 ICP interviews (parallel, Weeks 1–4):**** 10–20 problem interviews, pain not pitch.

Stage 2 — Deterministic queries that prove the graph · part of M3 *(Weeks 3–4)*

Ship deterministic queries **before** AI (FOUNDATION §"Immediate next steps": Q8 + Q4 first).

- ****CP-2.1 QueryPlan registry + ResultGraph contract.****
- ****CP-2.2 BlastRadius (Q8)**** — reverse transitive closure, ranked, paths included. **The wedge.**
 - **Gate:* blast radius of a core module matches hand-derived dependents on a known repo.
- ****CP-2.3 Centrality (Q4), Dependencies (Q9), Risk-lite (Q10:** complexity×fan-in×churn).**
 - **Gate:* top-5 important modules match a knowledgeable judge.
- ****CP-2.4 Entrypoints (Q3), Modules (Q2 structural).****
 - **Gate:* entrypoints correct on FastAPI/Flask/CLI repos.

Stage 3 — Semantic layer · M2, AI at the edge *(Weeks 4–5)*

- ****CP-3.1 RAG context assembler**** — graph-selected subgraph, token-budgeted; never send the repo (ARCHITECTURE §"RAG rule").
- ****CP-3.2 Module/function summaries****, cached by ``content_hash``, ``SemanticAnnotation.derived_from`` populated.
 - **Gate:* unchanged file → provable zero second LLM call.
- ****CP-3.3 Embeddings + concept search (Q7)**** over name+docstring+summary.
 - **Gate:* "where is X" returns correct nodes when X isn't literally named.
- ****CP-3.4 Narrated answers**** for Q1, Q6 (learning path), Q8 story; every claim clickable to a real node.
 - *****MILESTONE GATE M2+M3:***** the 10 launch questions answer end-to-end; ****8/10 correct on an unfamiliar repo**** judged by someone who knows it; every narrated claim traces to a node.

Stage 4 — The Hero Moment · frontend + EXPERIENCE Phase 1 *(Weeks 5–7)*

- ****CP-4.1 ViewSpec compiler**** (server-side: nodes-at-zoom, clusters, precomputed layout, risk colors, choreography).
- ****CP-4.2 Next.js + Graph State Manager + WebGL renderer**** (sigma.js/cosmos class); semantic zoom L1–3.

- **CP-4.3** Assembly reveal + understanding-language pipeline copy; dark, screenshot-worthy default.
- **CP-4.4** Click-to-story w/ synchronized highlight + the ripple (blast-radius animation).
 - **EXPERIENCE GATE**: 10-second silent test → "wait, what is this?" not "nice graph tool"; 60fps on a 13" laptop; every visual claim clickable to code.

Stage 5 — The validation gate · M4 (Weeks 7–8) — the decision that governs everything after

- **CP-5.1** Live on real users' repos.
 - **KILL/CONTINUE GATE (STRATEGY §7)**: a real developer uses it on *their* repo and says "I'd use this again" unprompted. If <5 of ~20 ICPs pull, change the wedge before writing more code.
- **B-2** Pricing conversations: 10 real talks against the Free/Pro/Team hypothesis (STRATEGY §8).

Stage 6 — Productionization (post-validation, Weeks 8–12)

- **CP-6.1** Deploy infra (containerize, managed Postgres for *metadata/jobs only*, object storage for graphs, one region). Introduce a background worker only if needed — prefer FastAPI BackgroundTasks/`arq` before Celery.
- **CP-6.2** Auth (GitHub OAuth), accounts, private repos, job queue with live progress (pipeline checkpoints feed the EXPERIENCE progress UI — one source for theater + telemetry).
- **CP-6.3** Billing (Stripe): Free / Pro / Team.
- **CP-6.4** Observability, error tracking, backups, secrets, security review.
 - **Gate**: a stranger signs up, pays, analyzes a private repo, gets a result — zero manual intervention.

Stage 7 — Distribution engine · L3 (Weeks 10–16, overlaps)

- **CP-7.1** Free public-repo analysis + shareable map links ("Share this view").
- **CP-7.2** CodeLens Score + methodology page.
- **CP-7.3** README badge (`CodeLens: A-`).
- **CP-7.4** Editorial engine — "we mapped [famous repo]" posts (content + SEO + badge loop).
 - **Gate**: first inbound signup traced to a shared map or badge (organic distribution proven).

Stage 8 — The credibility engine · L2 / Phase-Trust (Weeks 14–20)

- **CP-8.1** GitHub App — record a blast-radius prediction per merged PR.
- **CP-8.2** Outcome watcher — CI fails, reverts, hotfixes touching predicted files within N days (conservative signals only).
- **CP-8.3** Accuracy Ledger — private dashboard → **public** the moment numbers are honest & stable.
 - **MOAT-START GATE**: publish the first honest accuracy number. "We tracked our predictions for a month — here's our error rate."

Stage 9 — Scale & platform *(post-traction, 6–18 months)*

- **CP-9.1** JS/TS parser to parity. **CP-9.2** Storage graduation (Neo4j/Qdrant/Redis) — *only when load justifies each*, migrated mechanically behind `GraphStore`. **CP-9.3** Architect tier (coupling, debt ranking, boundaries, continuous monitoring). **CP-9.4** Team features + Jira/Linear + more languages. **CP-9.5** Fundraise-or-bootstrap decision — the Ledger + traction *is* the deck.

Cross-cutting sections the doc will also include

- **"The one gate that matters right now" callout: everything before CP-5.1 exists to reach that unprompted "I'd use this again." Stages 6–9 are forbidden to start until it passes.
- **Critical path** diagram-in-text: CP-0 → CP-1 → CP-2 → CP-3 → CP-4 → **CP-5.1 gate** → CP-6/7/8 (parallelizable) → CP-9. Business track (B-0, B-1, B-2) runs alongside CP-0...CP-5.
- **Constitution guardrails restated as checklist items** on relevant checkpoints (evidence before explanation; deterministic first; incremental by hash; confidence visible; the Ledger never lies) — from ROADMAP §Constitution.

Verification

Since the deliverable is a document, verification is a review pass:

1. Re-read `PLANNING/CHECKPOINTS.md` end-to-end after writing; confirm every checkpoint has all three of Deliverable / Gate / Why.
2. Cross-check each milestone gate against its source (ARCHITECTURE §Milestones M1–M5, STRATEGY §7 kill/continue, EXPERIENCE §The test, FOUNDATION §10 questions) — no gate should be softer than the founding doc's.
3. Confirm CP-0 accurately names the two real drifts (`config.py`, `requirements.txt` incl. missing `networkx`) against the current files.
4. Confirm no code was modified (git status shows only the new doc + this plan).